

claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf\_416cc0e5-a35\agent-aueb11dd7f571f782.jsonl`
- SHA-256: `25b54b5b4728d358bdab0cbcc3be9d845de7eadf69ecb348b88fae9ad49a1b79`
- Source modified: `2026-06-22T09:50:23+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf\_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:49:15.012Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:

```
{
  "title": "CRITICAL: trajectory_hybrid.process_video() docstring at line 141-143 ADMITS the type contract gap",
  "file": "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/trajectory_hybrid.py:138-143",
  "severity": "high",
  "detail": "Line 141-143 comment: 'override-ignore: the ABC declares the Result contract (Success|Failure) but every live segmenter still returns a bare list ? the documented deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).' This is a KNOWN gap. The return statement at line 379 returns a bare list, not Success/Failure. The nightly runner line 349 fallthrough 'result.value if hasattr(result, \"value\") else result' masks this, but the type system is lying. The docstring is candid: this is a 'deliberate-incremental gap' ? not a bug per se, but a contract debt.",
  "fix": "This is documented as intentional. No immediate fix required for nightly_reelcount.py since it handles both cases (line 349). However, trajectory_hybrid.py return type should be corrected long-term to match motion.py's future (Success | Failure)."
```

2. assistant (2026-06-22T09:49:17.070Z)

I'll systematically review the nightly e2e regression engine files and adversarially verify the trajectory_hybrid finding. Let me start by reading all the relevant files.

3. user (2026-06-22T09:49:18.661Z)

```
1      """Email the nightly regression report ? stdlib SMTP, creds from env or GCP Secret
Manager.
2
3      Used by ``scripts/nightly_reelcount.py --email`` (and reusable by any nightly).
Deliberately
```

```

4      dependency-light (stdlib ``smtplib`` + ``email.message``) and **graceful**: if no SMTP
password can
5      be resolved it logs + returns ``False`` (the report is still written/printed ? a broken
mailer must
6      never mask a regression).
7
8      Credential resolution (first hit wins), so it works on a GCP VM, locally, or in tests:
9      ? Recipient: ``--email-to`` arg ? ``NIGHTLY_EMAIL_TO`` ? ``avi.dullu@gmail.com``
10     ? SMTP host: ``NIGHTLY_SMTP_HOST`` (default ``smtp.gmail.com``) . port
``NIGHTLY_SMTP_PORT`` (587, STARTTLS)
11     ? SMTP user: ``NIGHTLY_SMTP_USER`` (default = recipient)
12     ? SMTP pass: ``NIGHTLY_SMTP_PASS`` ? Secret Manager ``NIGHTLY_SMTP_SECRET``
13     (`projects/<p>/secrets/<name>/versions/latest``; via the google-cloud-
secret-manager
14     lib if installed, else the ``gcloud`` CLI). A Gmail **app password** is
the simplest.
15
16     Set the secret once (owner): echo -n '<app-password>' | gcloud secrets create nightly-
smtp-pass --data-file=-
17     and on the VM: export NIGHTLY_SMTP_SECRET=projects/<proj>/secrets/nightly-smtp-
pass/versions/latest
18     export NIGHTLY_SMTP_USER=<your-gmail>
NIGHTLY_EMAIL_TO=avi.dullu@gmail.com
19     ""
20     from __future__ import annotations
21
22     import logging
23     import os
24     from email.message import EmailMessage
25     from typing import Optional
26
27     log = logging.getLogger("nightly_email")
28
29     DEFAULT_TO = "avi.dullu@gmail.com"
30     DEFAULT_HOST = "smtp.gmail.com"
31     DEFAULT_PORT = 587
32
33
34     def build_message(subject: str, body_md: str, from_addr: str, to_addr: str) ->
EmailMessage:
35     """A text/plain email carrying the markdown report (renders fine in any client).
Pure ? testable."""
36     msg = EmailMessage()
37     msg["Subject"] = subject
38     msg["From"] = from_addr
39     msg["To"] = to_addr
40     msg.set_content(body_md)
41     return msg
42
43
44     def resolve_secret(env: Optional[dict] = None) -> Optional[str]:
45     """Resolve the SMTP password from env or GCP Secret Manager. Returns None if
unresolvable."""
46     env = env if env is not None else os.environ
47     direct = env.get("NIGHTLY_SMTP_PASS")
48     if direct:
49         return direct
50     secret_ref = env.get("NIGHTLY_SMTP_SECRET")
51     if not secret_ref:
52         return None
53     # Prefer the client lib; fall back to the gcloud CLI (no extra dependency required).
54     try:
55         from google.cloud import secretmanager # type: ignore
56         client = secretmanager.SecretManagerServiceClient()
57         return

```

```

client.access_secret_version(name=secret_ref).payload.data.decode("utf-8")
58     except Exception: # noqa: BLE001 ? lib absent or auth issue ? try the CLI
59         pass
60     try:
61         import subprocess
62         name = secret_ref.split("/secrets/", 1)[1].split("/", 1)[0] if "/secrets/" in
secret_ref else secret_ref
63         out = subprocess.run(["gcloud", "secrets", "versions", "access", "latest", "--
secret", name],
64                               capture_output=True, text=True, timeout=30)
65         return out.stdout.strip() if out.returncode == 0 and out.stdout.strip() else
None
66     except Exception as e: # noqa: BLE001
67         log.warning("secret-manager resolution failed: %s", e)
68         return None
69
70
71     def send_report(subject: str, body_md: str, to: Optional[str] = None,
72                   env: Optional[dict] = None) -> bool:
73         """Send the report. Returns True if sent, False if skipped (no creds) ? never raises
on missing creds."""
74         env = env if env is not None else os.environ
75         to_addr = to or env.get("NIGHTLY_EMAIL_TO") or DEFAULT_TO
76         host = env.get("NIGHTLY_SMTP_HOST", DEFAULT_HOST)
77         port = int(env.get("NIGHTLY_SMTP_PORT", DEFAULT_PORT))
78         user = env.get("NIGHTLY_SMTP_USER") or to_addr
79         password = resolve_secret(env)
80         if not password:
81             log.warning("no SMTP password (set NIGHTLY_SMTP_PASS or NIGHTLY_SMTP_SECRET) ?
email skipped.")
82             return False
83
84         import smtplib
85         msg = build_message(subject, body_md, from_addr=user, to_addr=to_addr)
86         with smtplib.SMTP(host, port, timeout=60) as s:
87             s.starttls()
88             s.login(user, password)
89             s.send_message(msg)
90         log.info("emailed report to %s", to_addr)
91         return True
92

```

4. user (2026-06-22T09:49:18.969Z)

```

1     """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM
nightly).
2
3     Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a
deployment serves
4     it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does
not change**
5     vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
of CACHED
6     trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
model*, so it
7     needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8     owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
9
10    What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11    ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12    Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so

```

```

this is asserted
13      **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see $guard).
14      ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15      ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16      ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18      Extensible pattern (add a video = one manifest row + re-freeze):
19      ``eval_baselines/reelcount_manifest.json`` ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20      ``eval_baselines/reelcount_baseline.json`` ? the frozen expected counts (regenerated,
never hand-edited)
21
22      Usage
23      -----
24      Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25      python scripts/nightly_reelcount.py --update-baseline
26
27      Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28      python scripts/nightly_reelcount.py --email
29
30      Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31      (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32      """
33      from __future__ import annotations
34
35      import argparse
36      import datetime as _dt
37      import json
38      import os
39      import sys
40      from dataclasses import dataclass, field
41      from typing import Any, Dict, List, Optional, Tuple
42
43      REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
44      if REPO_ROOT not in sys.path:
45          sys.path.insert(0, REPO_ROOT)
46
47      # billing is pure (typing only) ? safe to import at module top.
48      from backend import billing # noqa: E402
49
50      DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
51      DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
52      DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
53
54      #: Quality metrics we report + (when labels exist) gate ? higher is better, small
tolerance.
55      QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
56      DEFAULT_QUALITY_TOL = 0.02      # quality is float/labeled ? a looser tripwire than the
exact reel-count
57      DEFAULT_FX_INR_PER_USD = 83.0
58      #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-
south1, ~mid-2026).
59      DEFAULT_MACHINE_USD_PER_HR = 0.35
60
61
62      @dataclass(frozen=True)
63      class Tolerances:

```

```

64     quality_tol: float = DEFAULT_QUALITY_TOL
65
66     def to_dict(self) -> Dict[str, Any]:
67         return {"quality_tol": self.quality_tol}
68
69
70     # ----- #
71     # Pure cost math (wraps backend.billing ? no IO).
72     # ----- #
73     def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74                    fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75         """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
arithmetic).
76
77         ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
``--vm-uptime-seconds``;
78         otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
for boot/install)."""
79         rate_per_sec = float(machine_usd_per_hr) / 3600.0
80         usage = {billing.WALL_SECONDS: float(billed_seconds)}
81         rates = {billing.WALL_SECONDS: rate_per_sec}
82         usd, breakdown = billing.compute_cost_usd(usage, rates)
83         return {
84             "billed_seconds": round(float(billed_seconds), 1),
85             "gpu_seconds": round(float(gpu_seconds), 1),
86             "machine_usd_per_hr": float(machine_usd_per_hr),
87             "usd": usd,
88             "inr": billing.to_inr(usd, fx_inr_per_usd),
89             "breakdown": breakdown,
90         }
91
92
93     # ----- #
94     # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
95     # ----- #
96     def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
97                          cost: Dict[str, Any]) -> Dict[str, Any]:
98         """Compact, comparable snapshot from per-video run results.
99
100         Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
wall_seconds}``.
101         A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
never hides it)."""
102         per_video: Dict[str, Any] = {}
103         for r in run_results:
104             per_video[r["name"]] = {
105                 "reel_count": r.get("reel_count"),
106                 "ok": bool(r.get("ok", False)),
107                 "error": r.get("error"),
108                 "quality": r.get("quality"),
109             }
110         return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
111               "n": len(per_video)}
112
113
114     @dataclass
115     class Finding:
116         video: str
117         metric: str # "reel_count" | "error" | a quality key | "segmenter" |
"corpus"
118         kind: str # "regression" | "improvement" | "stale"
119         baseline: Optional[float] = None
120         current: Optional[float] = None
121         detail: str = ""
122

```

```

123     def message(self) -> str:
124         if self.kind == "stale":
125             return f"[STALE] {self.metric}: {self.detail}"
126         if self.metric == "error":
127             return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
128         b = "?" if self.baseline is None else f"{self.baseline:g}"
129         c = "?" if self.current is None else f"{self.current:g}"
130         tag = "REGRESSION" if self.kind == "regression" else "improvement"
131         return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"
132
133
134     @dataclass
135     class NightlyResult:
136         findings: List[Finding] = field(default_factory=list)
137         added: List[str] = field(default_factory=list)
138         removed: List[str] = field(default_factory=list)
139
140         @property
141         def regressions(self) -> List[Finding]:
142             return [f for f in self.findings if f.kind == "regression"]
143
144         @property
145         def improvements(self) -> List[Finding]:
146             return [f for f in self.findings if f.kind == "improvement"]
147
148         @property
149         def stale(self) -> List[Finding]:
150             return [f for f in self.findings if f.kind == "stale"]
151
152         def overall_status(self) -> str:
153             if self.regressions:
154                 return "REGRESSION"
155             if self.stale:
156                 return "STALE"
157             return "PASS"
158
159         def exit_code(self) -> int:
160             if self.regressions:
161                 return 1
162             if self.stale:
163                 return 4
164             return 0
165
166
167     def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) ->
NightlyResult:
168         """Diff current run vs frozen baseline.
169
170         * Segmenter changed ? STALE (numbers not comparable; re-freeze).
171         * Corpus (video set) changed ? STALE + report added/removed; still diff the
INTERSECTION.
172         * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
regression;
173         quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
174         """
175         res = NightlyResult()
176         if baseline.get("segmenter") != current.get("segmenter"):
177             res.findings.append(Finding("", "segmenter", "stale",
178                                     detail=f"{baseline.get('segmenter')} ?
{current.get('segmenter')}"))
179             return res
180
181         b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
182         res.added = sorted(set(c_pv) - set(b_pv))
183         res.removed = sorted(set(b_pv) - set(c_pv))

```

```

184         if res.added or res.removed:
185             res.findings.append(Finding("", "corpus", "stale",
186                                     detail=f"added={res.added} removed={res.removed}"))
187
188     for v in sorted(set(b_pv) & set(c_pv)):
189         b, c = b_pv[v], c_pv[v]
190         # 1) pipeline must still succeed
191         if not c.get("ok", False) or c.get("reel_count") is None:
192             res.findings.append(Finding(v, "error", "regression",
193                                     detail=str(c.get("error") or "no reel_count
194 produced"))))
195             continue
196         # 2) reel count ? EXACT
197         bc, cc = b.get("reel_count"), c.get("reel_count")
198         if bc is not None and cc is not None and cc != bc:
199             res.findings.append(Finding(v, "reel_count", "regression", float(bc),
200 float(cc)))
201         # 3) quality metrics ? tolerance (only if both sides have them)
202         bq, cq = b.get("quality") or {}, c.get("quality") or {}
203         for m in QUALITY_HIGHER:
204             bv, cv = bq.get(m), cq.get(m)
205             if bv is None or cv is None:
206                 continue
207             if cv < bv - tols.quality_tol:
208                 res.findings.append(Finding(v, m, "regression", bv, cv))
209             elif cv > bv + tols.quality_tol:
210                 res.findings.append(Finding(v, m, "improvement", bv, cv))
211     return res
212
213 # ----- #
214 # Report formatting (pure).
215 # ----- #
216 def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
217     if not qd or qd.get(key) is None:
218         return "?"
219     return f"{qd[key]:.3f}"
220
221 def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
222                  result: NightlyResult, date: str, head: str) -> str:
223     status = result.overall_status()
224     badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
225             "STALE": "? STALE (re-freeze the baseline)"}[status]
226     cost = current.get("cost", {})
227     L: List[str] = [
228         f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
229         "",
230         f"***Status: {badge}** . exit code {result.exit_code()} . segmenter
231         `{current.get('segmenter')}`",
232         "",
233         f"- HEAD `{head}` . baseline `{baseline.get('git_commit', '?')}` (frozen
234         {baseline.get('generated_at', '?')})",
235         f"- **Run cost: ${cost.get('usd', 0):.4f} ({cost.get('inr', 0):.2f})** . "
236         f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
237 0)}/hr",
238         "",
239     ]
240     if result.regressions:
241         L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions]
242     + [""]
243     if result.stale:
244         L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in
245 result.stale]
246     L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after

```

```

confirming the change is intended).", ""]
242
243     # Per-video table.
244     b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
245     L += ["## Per-video", "",
246          "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
247          "|---|---|---|---|---|"]
248     for v in sorted(set(b_pv) | set(c_pv)):
249         b, c = b_pv.get(v, {}), c_pv.get(v, {})
250         bc = b.get("reel_count")
251         cc = c.get("reel_count")
252         rc = f"'?' if bc is None else bc?{'ERROR' if not c.get('ok', True) else ('?'
if cc is None else cc)}"
253         bq, cq = b.get("quality"), c.get("quality")
254         tf = f"_{q(bq, 'tol_f1')}?_{q(cq, 'tol_f1')}"
255         f5 = f"_{q(bq, 'f1@0.5')}?_{q(cq, 'f1@0.5')}"
256         vstatus = "?" if any(f.video == v and f.kind == "regression" for f in
result.findings) else "?"
257         L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
258     L.append("")
259
260     if result.improvements:
261         L += ["_Improvements over baseline (re-freeze to ratchet up):_" \
262              + [f"- {f.message()}" for f in result.improvements] + [""]
263     L += ["---",
264          "_Full configured+checkpointed pipeline on the real model (GCP GPU VM).
Tripwire only ? does "
265          "not change the promotion gate. Complements PR #202's cached-trajectory CPU
re-score._"]
266     return "\n".join(L)
267
268
269     # ----- #
270     # IO shell ? running the configured pipeline per video (needs backend + GPU + video).
271     # ----- #
272     def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
273         """Load ground-truth rally intervals from a golden-features JSON (`gts` key) or a
rallies CSV
274         (`start,end` columns / pairs). Returns None when no labels are configured."""
275         if not labels_ref:
276             return None
277         from backend.storage import fetch
278         local = fetch(labels_ref)
279         if local.endswith(".json"):
280             with open(local, encoding="utf-8") as fh:
281                 data = json.load(fh)
282                 return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
283         # CSV: header start,end (or first two numeric columns)
284         import csv
285         out: List[Tuple[float, float]] = []
286         with open(local, encoding="utf-8") as fh:
287             for row in csv.reader(fh):
288                 try:
289                     out.append((float(row[0]), float(row[1])))
290                 except (ValueError, IndexError):
291                     continue
292         return out
293
294
295     def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
296         """(`start, end` pairs from segmenter result dicts, tolerant of the two key
conventions in the
297         codebase (`start_time`/`end_time` ? motion/DB shape ? or `start`/`end`)).
Used only for the
298         optional quality metrics; the reel COUNT is `len(segs)` regardless, so a key

```

```

mismatch degrades
299     quality scoring gracefully (skipped) without ever miscounting reels."""
300     out: List[Tuple[float, float]] = []
301     for s in segs:
302         start = s.get("start_time", s.get("start"))
303         end = s.get("end_time", s.get("end"))
304         if start is not None and end is not None:
305             out.append((float(start), float(end)))
306     return out
307
308
309     def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
310                      rerun_on_mismatch: bool = True,
311                      baseline_count: Optional[int] = None) -> Dict[str, Any]:
312         """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.
313
314         The re-run-once guard (the one known non-determinism ? a transient
`add_play_segment`?None drop,
315         database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
316         reproduce, a real change will."""
317         import random
318         import time
319
320         from backend.eval import rally_seg_eval
321         from backend.infrastructure.database import Database
322         from backend.pipeline.segmenters import segmenter_registry
323         from backend.results import Failure
324         from backend.storage import fetch
325         from backend.utils.hashing import compute_video_id
326
327         name = video["name"]
328         try:
329             local = fetch(video["uri"])
330         except Exception as e: # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
332                    "quality": None, "wall_seconds": 0.0}
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
338                    "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}
339
340         def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341             random.seed(0) # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342             db = Database(db_path)
343             video_id = compute_video_id(local)
344             t0 = time.monotonic()
345             result = seg_cls(db, config).process_video(video_id, local)
346             wall = time.monotonic() - t0
347             if isinstance(result, Failure):
348                 return None, None, wall, getattr(result, "message", "segmenter failed")
349             segs = result.value if hasattr(result, "value") else result
350             intervals = _segments_to_intervals(segs)
351             return len(segs), intervals, wall, None
352
353         import tempfile
354         total_wall = 0.0
355         with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:

```

```

356         count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357         total_wall += wall
358         if err is not None:
359             return {"name": name, "reel_count": None, "ok": False, "error": err,
360                    "quality": None, "wall_seconds": round(total_wall, 2)}
361         # re-run-once guard for the transient DB-drop flake
362         if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
363             count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
364             total_wall += wall2
365             if err2 is None and count2 is not None:
366                 count, intervals = count2, intervals2 # the reproduced (or corrected)
value
367
368         quality = None
369         if gts and intervals is not None:
370             row = rally_seg_eval.score_intervals(intervals, gts)
371             quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
"tol_precision", "tol_recall")}
372                 if k in row}
373
374         return {"name": name, "reel_count": count, "ok": True, "error": None,
375                "quality": quality, "wall_seconds": round(total_wall, 2)}
376
377
378     def load_manifest(path: str) -> Dict[str, Any]:
379         with open(path, encoding="utf-8") as fh:
380             return json.load(fh)
381
382
383     def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
384                   rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
385         """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
386         from backend.config import load_config
387
388         config = load_config()
389         # apply manifest config overrides onto the typed config (dotted keys)
390         for dotted, val in (manifest.get("config_overrides") or {}).items():
391             _set_dotted(config, dotted, val)
392         segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
393         b_pv = (baseline or {}).get("per_video", {})
394         results: List[Dict[str, Any]] = []
395         total_wall = 0.0
396         for v in manifest.get("videos", []):
397             base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
398             r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
399             total_wall += float(r.get("wall_seconds") or 0.0)
400             results.append(r)
401         return results, total_wall
402
403
404     def _set_dotted(config: Any, dotted: str, value: Any) -> None:
405         """Best-effort dotted-path set onto a pydantic config (e.g.
'indexing.motion_threshold')."""
406         parts = dotted.split(".")
407         obj = config
408         for p in parts[:-1]:
409             obj = getattr(obj, p, None)
410             if obj is None:
411                 return
412         try:
413             setattr(obj, parts[-1], value)
414         except Exception: # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't
crash the run

```

```

415         pass
416
417
418     def _git_head() -> str:
419         import subprocess
420         try:
421             out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
422                                 capture_output=True, text=True, timeout=10)
423             return out.stdout.strip() or "?"
424         except Exception: # noqa: BLE001
425             return "?"
426
427
428     def load_baseline(path: str) -> Optional[Dict[str, Any]]:
429         if not os.path.exists(path):
430             return None
431         with open(path, encoding="utf-8") as fh:
432             return json.load(fh)
433
434
435     def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
436         out = dict(snapshot)
437         out["generated_at"] = _dt.date.today().isoformat()
438         out["git_commit"] = _git_head()
439         out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python
scripts/nightly_reelcount.py "
440                       "--update-baseline` after an INTENDED engine change or when adding a
video. Tied to the "
441                       "configured+checkpointed engine + the manifest corpus.")
442         out.pop("cost", None) # cost is per-run, not part of the frozen baseline
443         os.makedirs(os.path.dirname(path), exist_ok=True)
444         tmp = path + ".tmp"
445         with open(tmp, "w", encoding="utf-8") as fh:
446             json.dump(out, fh, indent=2, sort_keys=True)
447             fh.write("\n")
448         os.replace(tmp, path)
449
450
451     def build_parser() -> argparse.ArgumentParser:
452         p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count +
quality + cost.")
453         p.add_argument("--manifest", default=DEFAULT_MANIFEST)
454         p.add_argument("--baseline", default=DEFAULT_BASELINE)
455         p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
456         p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
457         p.add_argument("--vm-uptime-seconds", type=float, default=None,
458                       help="authoritative VM uptime (boot?delete) for the cost line; "
459                       "defaults to the in-process pipeline wall time (a lower bound)")
460         p.add_argument("--no-rerun-guard", action="store_true",
461                       help="disable the re-run-once-on-mismatch guard (the DB-drop flake
mitigation)")
462         p.add_argument("--update-baseline", action="store_true",
463                       help="freeze the current reel counts + quality as the new baseline
instead of checking")
464         p.add_argument("--email", action="store_true", help="email the report (see
scripts/nightly_email.py)")
465         p.add_argument("--email-to", default=None, help="override the report recipient")
466         p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
467         p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
468         return p
469
470
471     def main(argv: Optional[List[str]] = None) -> int:

```

```

472     import logging
473     logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
474     log = logging.getLogger("nightly_reelcount")
475     args = build_parser().parse_args(argv)
476     tols = Tolerances(quality_tol=args.quality_tol)
477     date = args.date or _dt.date.today().isoformat()
478
479     if not os.path.exists(args.manifest):
480         log.error("manifest not found: %s", args.manifest)
481         return 2
482     manifest = load_manifest(args.manifest)
483     baseline = load_baseline(args.baseline)
484
485     try:
486         run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
487         except Exception as e: # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
488             log.error("run failed: %s", e)
489             return 2
490     if not run_results:
491         log.error("no videos in manifest %s", args.manifest)
492         return 2
493
494     billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
495     cost = cost_report(
496         billed_seconds=billed,
497         machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
498         fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
499         gpu_seconds=total_wall,
500     )
501     segmenter = manifest.get("segmenter", "motion")
502     snapshot = summarize_results(run_results, segmenter, cost)
503     snapshot["git_commit"] = _git_head()
504     snapshot["generated_at"] = date
505
506     if args.update_baseline:
507         save_baseline(args.baseline, snapshot)
508         log.info("baseline frozen ? %s", args.baseline)
509         for r in run_results:
510             log.info(" %s: reels=%s ok=%s", r["name"], r.get("reel_count"),
r.get("ok"))
511         return 0
512
513     if baseline is None:
514         log.error("no baseline at %s ? freeze one with --update-baseline (on the
configured engine).", args.baseline)
515         return 3
516
517     result = compare(baseline, snapshot, tols)
518     report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])
519
520     if not args.no_report:
521         os.makedirs(args.report_dir, exist_ok=True)
522         md_path = os.path.join(args.report_dir, f"{date}.md")
523         with open(md_path, "w", encoding="utf-8") as fh:
524             fh.write(report + "\n")
525         with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8")
as fh:
526             json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
527                        "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},

```

```

528             fh, indent=2, sort_keys=True)
529         log.info("report ? %s", md_path)
530
531     print(report)
532
533     if args.email:
534         try:
535             from scripts.nightly_email import send_report
536             subject = f"[KheISutra nightly] {result.overall_status()} ? reel-count
engine regression {date}"
537             sent = send_report(subject, report, to=args.email_to)
538             log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
539         except Exception as e: # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict
540             log.error("email failed (report still written): %s", e)
541
542     code = result.exit_code()
543     if code == 1:
544         log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions),
args.report_dir)
545     elif code == 4:
546         log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-
baseline")
547     else:
548         log.info("nightly reel-count check PASSED (cost $%.4f / ?%.2f)", cost["usd"],
cost["inr"])
549     return code
550
551
552 if __name__ == "__main__":
553     raise SystemExit(main())
554

```

5. user (2026-06-22T09:49:19.622Z)

```

1     #!/usr/bin/env bash
2     # Startup-script for the NIGHTLY engine-regression VM. Runs UNATTENDED on boot, then
DELETES the VM
3     # (cost rule: a running GPU VM is the only real cost). Survives SSH drops (it's a
startup-script, not
4     # an interactive session). Reads its config from instance metadata (set by launch.sh).
5     #
6     # Flow: Ops Agent ? pull repo @ HEAD from GCS ? build venv + WASB env ? stage
weights ?
7     #         run scripts/nightly_reelcount.py on the gs:// golden clips (emails the report)
?
8     #         upload report to GCS ? SELF-DELETE (even on failure, via the EXIT trap).
9     #
10    # Metadata keys (launch.sh sets these): repo-tgz-uri, git-sha, gcs-bucket, report-
prefix,
11    # weights-uri, email (true/false). NOT verified in CI ? dry-run via launch.sh on a
real VM first.
12    set -uo pipefail
13    exec > >(tee -a /var/log/nightly-regression.log) 2>&1
14    echo "==== nightly-regression startup $(date -u +%FT%TZ) ====="
15    BOOT_EPOCH=$(date +%s)
16
17    md() { curl -s -H "Metadata-Flavor: Google"
"http://metadata.google.internal/computeMetadata/v1/$1"; }
18    NAME=$(md instance/name)
19    ZONE=$(md instance/zone | awk -F/ '{print $NF}')
20    REPO_TGZ=$(md instance/attributes/repo-tgz-uri || true)
21    GIT_SHA=$(md instance/attributes/git-sha || echo unknown)
22    BUCKET=$(md instance/attributes/gcs-bucket || true)

```

```

23     REPORT_PREFIX=$(md instance/attributes/report-prefix || echo nightly/reports)
24     WEIGHTS_URI=$(md instance/attributes/weights-uri || true)
25     DO_EMAIL=$(md instance/attributes/email || echo true)
26     SMTP_SECRET=$(md instance/attributes/smtp-secret || true)
27     SMTP_USER=$(md instance/attributes/smtp-user || true)
28     EMAIL_TO=$(md instance/attributes/email-to || echo avi.dullu@gmail.com)
29     DATE=$(date -u +%F)
30
31     # ALWAYS delete the VM on exit (success OR failure) ? never leave a billing GPU VM
behind.
32     cleanup() {
33         echo "==== self-delete $NAME ($ZONE) ====="
34         gcloud compute instances delete "$NAME" --zone="$ZONE" --quiet || \
35             echo "WARN: self-delete failed ? DELETE MANUALLY: gcloud compute instances delete
$NAME --zone=$ZONE -q"
36     }
37     trap cleanup EXIT
38
39     fail_email() { # best-effort failure alert (the report email won't have fired on an
early crash)
40         echo "RUN FAILED ? attempting a failure alert"
41         if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
42             cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
43                 NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
44                 python - <<PY || true
45         from scripts.nightly_email import send_report
46         send_report("[KhelSutra nightly] ERROR ? engine regression VM run failed ($DATE)",
47             "The nightly engine-regression run failed on the VM before producing a
report.\n"
48             "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial
log.\nGIT_SHA=$GIT_SHA", to="$EMAIL_TO")
49         PY
50         fi
51     }
52     trap 'fail_email' ERR
53
54     # 1) Observability (Ops Agent) ? same as create_gpu_vm.sh bakes in.
55     curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && \
56         sudo bash add-google-cloud-ops-agent-repo.sh --also-install || echo "WARN: ops-agent
install failed (non-fatal)"
57
58     # 2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses the
worker SA).
59     mkdir -p ~/rally && cd ~/rally
60     gcloud storage cp "$REPO_TGZ" /tmp/rally.tgz
61     tar -xzf /tmp/rally.tgz -C ~/rally
62     echo "$GIT_SHA" > ~/rally/GIT_SHA
63
64     # 3) Build the env + native WASB (provider-agnostic DO scripts, reused on GCP per
deploy/gcp/README §4).
65     sudo bash deploy/digitalocean/mount_scratch.sh 2>/dev/null || true
66     export TMPDIR=/scratch 2>/dev/null || true
67     ./deploy/digitalocean/bootstrap.sh --gpu
68     . .venv/bin/activate
69     pip install -e '[storage-gcs]'
70     bash deploy/digitalocean/gpu_setup.sh || echo "WARN: gpu_setup (WASB env) returned
nonzero ? continuing"
71
72     # 4) Stage the WASB checkpoint (gpu_setup does NOT fetch weights ? README §4).
73     if [ -n "$WEIGHTS_URI" ]; then
74         mkdir -p ~/models/WASB-SBDT/pretrained_weights
75         gcloud storage cp "$WEIGHTS_URI" ~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
76         export WASB_WEIGHTS=~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar

```

```

77     fi
78
79     # 5) Run the regression on the gs:// golden clips in the committed manifest. Cost = VM
uptime so far.
80     UPTIME=$(( $(date +%s) - BOOT_EPOCH ))
81     EMAIL_FLAG=""; [ "$DO_EMAIL" = "true" ] && EMAIL_FLAG="--email --email-to $EMAIL_TO"
82     set +e
83     NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
84     python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
85     RUN_RC=$?
86     set -e
87     echo "runner exit code: $RUN_RC (0 pass / 1 regression / 4 stale)"
88
89     # 6) Upload the dated report to GCS for the record (alongside the email).
90     if [ -n "$BUCKET" ] && [ -d output/nightly_reelcount ]; then
91         gcloud storage cp -r "output/nightly_reelcount/$DATE."*
"gs://$BUCKET/$REPORT_PREFIX/$DATE/" || \
92         gcloud storage cp -r output/nightly_reelcount "gs://$BUCKET/$REPORT_PREFIX/$DATE/"
|| true
93     fi
94
95     echo "==== nightly-regression done rc=$RUN_RC $(date -u +%FT%TZ) ====="
96     trap - ERR # disable the failure-email trap; the EXIT (self-delete) trap stays
97     # the EXIT trap deletes the VM
98

```

6. user (2026-06-22T09:49:20.262Z)

```

1     #!/usr/bin/env bash
2     # Launch ONE nightly engine-regression run on an ephemeral GCP GPU VM, then return. The
VM runs
3     # run_on_vm.sh as its startup-script (regression ? email ? SELF-DELETE), so this just
stages the repo
4     # and kicks the VM off. Driver-agnostic: call it from Cloud Scheduler?Cloud Run job, a
cron on any
5     # always-on box, the owner's machine, or a Claude routine. Needs gcloud auth + the repo
checkout.
6     #
7     # bash deploy/gcp/nightly_regression/launch.sh # L4, stages HEAD,
kicks the VM
8     # RALLY_GPU=t4 RALLY_REF=$(git rev-parse HEAD) bash
deploy/gcp/nightly_regression/launch.sh
9     #
10    # ? Run this MANUALLY once as a dry-run (watch the serial log) before wiring a scheduler
? the VM-side
11    # flow is NOT covered by CI. Tail: gcloud compute instances get-serial-port-output
<name> --zone=<z>.
12    set -uo pipefail
13
14    PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
15    BUCKET="${RALLY_BUCKET:-kheldutra-rally-corpus}"
16    SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
17    GPU="${RALLY_GPU:-l4}"
18    NAME="${RALLY_VM_NAME:-rally-nightly-$(date -u +%Y%m%d)}"
19    REF="${RALLY_REF:-HEAD}"
20    WEIGHTS_URI="${RALLY_WEIGHTS_URI:-gs://$BUCKET/weights/wasb_badminton_best.pth.tar}"
21    REPORT_PREFIX="${RALLY_REPORT_PREFIX:-nightly/reports}"
22    DO_EMAIL="${RALLY_EMAIL:-true}"
23    SMTP_SECRET="${NIGHTLY_SMTP_SECRET:-projects/$PROJECT/secrets/nightly-smtp-
pass/versions/latest}"
24    SMTP_USER="${NIGHTLY_SMTP_USER:-}"
25    EMAIL_TO="${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"
26    HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
27    STARTUP="$HERE/run_on_vm.sh"

```

```

28     export CLOUDSDK_CORE_DISABLE_PROMPTS=1
29
30     ZONES="${RALLY_GCP_ZONES:-asia-south1-a asia-south1-b asia-south1-c asia-southeast1-a
asia-southeast1-b asia-southeast1-c}"
31     if [ "$GPU" = "l4" ]; then MACHINE="g2-standard-4"; ACCEL=(); else
MACHINE="n1-standard-8"; ACCEL=(--accelerator=type=nvidia-tesla-t4,count=1); fi
32
33     # 1) Stage the repo @ REF to GCS (no GitHub auth on the VM; the worker SA can read the
bucket).
34     SHA="$(git rev-parse "$REF")"
35     TGZ_URI="gs://$BUCKET/nightly/repo/${SHA}.tgz"
36     echo "== staging repo $SHA -> $TGZ_URI =="
37     TMP_TGZ="$(mktemp --suffix=.tgz)"
38     git archive --format=tar.gz -o "$TMP_TGZ" "$REF"
39     gcloud storage cp "$TMP_TGZ" "$TGZ_URI" --project="$PROJECT"
40     rm -f "$TMP_TGZ"
41
42     # 2) Zone-sweep create the VM (mirrors create_gpu_vm.sh) with run_on_vm.sh + the config
metadata.
43     ERR="$(mktemp)"
44     for Z in $ZONES; do
45         echo "== creating $NAME ($GPU) in $Z =="
46         if gcloud compute instances create "$NAME" --project="$PROJECT" --zone="$Z" \
47             --machine-type="$MACHINE" "${ACCEL[@]}" \
48             --maintenance-policy=TERMINATE --provisioning-model=STANDARD \
49             --image-family=common-cu129-ubuntu-2204-nvidia-580 --image-project=deeplearning-
platform-release \
50             --boot-disk-size=120GB --boot-disk-type=pd-balanced \
51             --service-account="$SA" --scopes=cloud-platform \
52             --metadata-from-file=startup-script="$STARTUP" \
53             --metadata=repo-tgz-uri="$TGZ_URI",git-sha="$SHA",gcs-bucket="$BUCKET",report-
prefix="$REPORT_PREFIX",weights-uri="$WEIGHTS_URI",email="$DO_EMAIL",smtp-
secret="$SMTP_SECRET",smtp-user="$SMTP_USER",email-to="$EMAIL_TO" \
54             2>"$ERR"; then
55             echo "? launched $NAME in $Z ? it will run the regression, email $EMAIL_TO, and
self-delete."
56             echo "    serial log: gcloud compute instances get-serial-port-output $NAME --zone=$Z
--project=$PROJECT"
57             echo "    if it doesn't self-delete: gcloud compute instances delete $NAME --zone=$Z
-q"
58             rm -f "$ERR"; exit 0
59         fi
60         if grep -qiE "STOCKOUT|does not have enough" "$ERR"; then echo "    (no $GPU capacity in
$Z ? next zone)"; continue; fi
61         echo "    create failed:"; tail -4 "$ERR"; rm -f "$ERR"; exit 1
62     done
63     echo "? no $GPU capacity in any swept zone ($ZONES). Retry later or RALLY_GPU=t4."
64     rm -f "$ERR"; exit 1
65

```

7. user (2026-06-22T09:49:21.207Z)

```

1     #!/usr/bin/env bash
2     # Wire the nightly engine-regression as a CLOUD-NATIVE, scale-to-zero job:
3     #   Cloud Scheduler (cron) ? Cloud Run JOB (the launcher image) ? launch.sh
4     #   ? ephemeral GPU VM (regression + email + SELF-DELETE).
5     # The only always-on cost is $0 (Cloud Run jobs + Scheduler are scale-to-zero; the GPU
VM is ephemeral).
6     #
7     # ? OWNER-RUN, ONCE, after a successful manual `bash
deploy/gcp/nightly_regression/launch.sh` dry-run.
8     # NOT verified in CI (no GCP/GPU here). Prereqs: GPU quota (README §2), the golden clips
+ weights in
9     # gs://<bucket>, and the SMTP secret (see README "Enable").
10    #

```

```

11      #   bash deploy/gcp/nightly_regression/register_scheduler.sh           # build + deploy
+ schedule (03:00 IST)
12      #   RALLY_CRON='0 3 * * *' RALLY_TZ='Asia/Kolkata' bash ../register_scheduler.sh
13      set -euo pipefail
14
15      PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
16      REGION="${RALLY_RUN_REGION:-asia-south1}"
17      JOB="${RALLY_RUN_JOB:-nightly-regression-launcher}"
18      SCHED="${RALLY_SCHED_NAME:-nightly-regression}"
19      CRON="${RALLY_CRON:-0 3 * * *}"           # 03:00 daily
20      TZ="${RALLY_TZ:-Asia/Kolkata}"
21      SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
22      IMAGE="${RALLY_LAUNCHER_IMAGE:-${REGION}-docker.pkg.dev/${PROJECT}/rally/nightly-
launcher:latest}"
23      REPO_ROOT="$(git rev-parse --show-toplevel)"
24      export CLOUDSDK_CORE_DISABLE_PROMPTS=1
25
26      echo "== enable APIs (run, scheduler, build, artifact registry) =="
27      gcloud services enable run.googleapis.com cloudscheduler.googleapis.com \
28          cloudbuild.googleapis.com artifactregistry.googleapis.com --project="$PROJECT"
29
30      echo "== ensure the Artifact Registry repo 'rally' exists =="
31      gcloud artifacts repositories describe rally --location="$REGION" --project="$PROJECT"
>/dev/null 2>&1 || \
32      gcloud artifacts repositories create rally --repository-format=docker
--location="$REGION" --project="$PROJECT"
33
34      echo "== build + push the launcher image from the repo root =="
35      ( cd "$REPO_ROOT" && gcloud builds submit --project="$PROJECT" --tag "$IMAGE" \
36          --gcs-source-staging-dir="gs://${RALLY_BUILD_BUCKET:-kshelsutra-rally-
corpus}/cloudbuild" \
37          -f deploy/gcp/nightly_regression/Dockerfile.launcher . )
38
39      echo "== create/update the Cloud Run JOB (launcher needs compute-admin to create the GPU
VM) =="
40      gcloud run jobs deploy "$JOB" --project="$PROJECT" --region="$REGION" \
41          --image="$IMAGE" --service-account="$SA" --max-retries=0 --task-timeout=900s \
42          --set-env-vars="RALLY_GPU=${RALLY_GPU:-
l4},RALLY_EMAIL=true,NIGHTLY_EMAIL_TO=${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"
43
44      echo "== schedule it (Cloud Scheduler ? Run job admin API) =="
45      RUN_URI="https://${REGION}-
run.googleapis.com/apis/run.googleapis.com/v1/namespaces/${PROJECT}/jobs/${JOB}:run"
46      gcloud scheduler jobs describe "$SCHED" --location="$REGION" --project="$PROJECT"
>/dev/null 2>&1 && \
47      gcloud scheduler jobs delete "$SCHED" --location="$REGION" --project="$PROJECT"
--quiet || true
48      gcloud scheduler jobs create http "$SCHED" --project="$PROJECT" --location="$REGION" \
49          --schedule="$CRON" --time-zone="$TZ" \
50          --uri="$RUN_URI" --http-method=POST \
51          --oauth-service-account-email="$SA"
52
53      cat <<EOF
54
55      ? Scheduled '$SCHED' ($CRON $TZ) ? Cloud Run job '$JOB' ? launch.sh ? ephemeral GPU VM.
56      IAM the launcher SA needs (grant once if missing):
57          roles/compute.instanceAdmin.v1 (create/delete the GPU VM)
58          roles/iam.serviceAccountUser (attach $SA to the VM)
59          roles/run.invoker + roles/cloudscheduler.jobRunner (Scheduler ? Run)
60      Trigger now to test: gcloud run jobs execute $JOB --region=$REGION
--project=$PROJECT
61      Remove: gcloud scheduler jobs delete $SCHED --location=$REGION; gcloud
run jobs delete $JOB --region=$REGION
62      EOF
63

```

8. user (2026-06-22T09:49:21.852Z)

```
1      # Tiny launcher image for the nightly engine-regression Cloud Run JOB. It is NOT the GPU
workload ?
2      # it only stages the repo + creates the ephemeral GPU VM (which self-deletes). Scale-to-
zero: the job
3      # runs for a few seconds nightly, then exits. Base = google/cloud-sdk (gcloud + git
preinstalled).
4      FROM gcr.io/google.com/cloudsdktool/cloud-sdk:slim
5      WORKDIR /app
6      # The repo is the build context (run `gcloud builds submit` from the repo root, see
register_scheduler.sh).
7      COPY . /app
8      ENTRYPOINT ["bash", "deploy/gcp/nightly_regression/launch.sh"]
9
```

9. user (2026-06-22T09:49:22.467Z)

```
1      {
2      "_doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The
configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality
metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row +
re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated
baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths.
labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present =>
quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your
actual short (~5-6 min) OWNED, minors-free clips before enabling.",
3      "segmenter": "wasb_hybrid",
4      "detector_impl": "native",
5      "config_overrides": {
6      "indexing.default_segmenter": "wasb_hybrid",
7      "indexing.detector_impl": "native"
8      },
9      "machine_usd_per_hr": 0.70,
10     "fx_inr_per_usd": 83.0,
11     "videos": [
12     {
13         "name": "GX030094",
14         "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX030094.mp4",
15         "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/GX030094_golden_features.json"
16     },
17     {
18         "name": "Badminton_BXH_2",
19         "uri": "gs://khelsutra-rally-corpus/nightly/videos/Badminton_BXH_2.mp4",
20         "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/Badminton_BXH_2_golden_features.json"
21     },
22     {
23         "name": "GX010128",
24         "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX010128.mp4",
25         "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/GX010128_golden_features.json"
26     }
27     ]
28     }
29
```

10. user (2026-06-22T09:49:23.180Z)

```
1      """"Shared engine for trajectory-substrate hybrid segmenters.
2
3      tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4      standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5      now the single home for the whole pipeline shape:
```

```

6
7     healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8     -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27
28    from backend.pipeline.segmenters.base import BasePlaySegmenter
29    from backend.utils.video import get_video_duration, extract_video_chunk
30    from backend.utils.paths import output_base
31    from backend.sports import sport_registry
32    from backend.providers import provider_registry
33    from backend.pipeline.detectors.base import DetectorRunner
34
35    logger = logging.getLogger(__name__)
36
37
38    def _fmt_ts(seconds: float) -> str:
39        seconds = max(0, int(seconds))
40        h, rem = divmod(seconds, 3600)
41        m, s = divmod(rem, 60)
42        return f"{h:02d}:{m:02d}:{s:02d}"
43
44
45    class TrajectoryHybridSegmenter(BasePlaySegmenter):
46        """Trajectory-detector hybrid: precise candidate rallies + AI semantic
47        boundaries."""
48
49        #: config section under `indexing` (velocity_threshold lives there), the
50        #: output subdir for trajectories, and the metadata detector-tag prefix.
51        family: str = ""
52        #: human-readable label used in log lines.
53        display_label: str = ""
54
55        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
56        Any]]:
57            """Return (runner, detector-specific detection_params extras) for the default
58            (WSL) path."""
59            raise NotImplementedError
60
61        def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
62        Dict[str, Any]]:
63            """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
64            with a
65            native runner override this; the base refuses with a clear message (M3.5: WASB
66            only)."""
67            raise NotImplementedError(
68                f"detector_impl='native' is not supported for the "
69                f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
70                has a "

```

```

64         f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
65
66     def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
67         """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
68
69         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
70         ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
71         is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
72         in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
73         ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
74         (the WSL runner) so the default production path is unchanged
75         (``docs/DECOUPLED_COMPUTE``).
76         """
77         impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
78         if impl == "stub":
79             from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
80             logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
self.display_label or "trajectory")
81             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
82         if impl in ("native", "native_wasb", "wasb_native"):
83             logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
self.display_label or "trajectory")
84             return self._build_native_runner(idx_cfg)
85         return self._build_runner(idx_cfg)
86
87     @staticmethod
88     def _probe_fps_width(video_path: str) -> tuple:
89         """Return (fps, frame_width) for a video, with sensible fallbacks."""
90         cap = cv2.VideoCapture(video_path)
91         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
92         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
93         cap.release()
94         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
95
96     @staticmethod
97     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
98         """Provider-reported exchange count, else a duration-based estimate.
99
100         One canonical implementation ? the twins had drifted (wasb relied on
101         ``int(None)`` raising into the fallback; same result, by accident).
102         """
103         shot_count = segment.get("shuttle_exchange_count")
104         if shot_count is None:
105             return max(3, int(duration / 0.8))
106         try:
107             return int(shot_count)
108         except (ValueError, TypeError):
109             return max(3, int(duration / 0.8))
110
111     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
112     -----
113
114     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
frame_width: float, video_path: str,
115                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
116         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE

```

```

118         returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
119         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
120         features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
121         return {
122             "peak_velocity": cw["peak_velocity"],
123             "mean_velocity": cw["mean_velocity"],
124             "active_frame_count": cw["active_frame_count"],
125             "track_id_count": cw["track_id_count"],
126         }
127
128     def _select_for_handoff(self, windows: List[Dict[str, Any]],
129                           window_stats: List[Dict[str, Any]],
130                           idx_cfg: Dict[str, Any]) -> List[int]:
131         """Indices of the candidate windows that proceed to the AI handoff (and thus may
132         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
133         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
134         Persisted candidates are NOT affected ? they remain the full superset for
offline
135         re-scoring."""
136         return list(range(len(windows)))
137
138     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
139                      player_pool: Optional[List[str]] = None,
140                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
141         # override-ignore: the ABC declares the Result contract (Success|Failure)
142         # but every live segmenter still returns a bare list ? the documented
143         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
144         label = self.display_label
145         # --- Sport profile + AI provider wiring (identical across segmenters) ---
146         sport_name = self.config.get("sport", "badminton")
147         try:
148             sport_profile = sport_registry.get(sport_name)
149         except KeyError as e:
150             logger.error(str(e))
151         return []
152
153         idx_cfg = self.config.get("indexing", {})
154         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
155         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
156         # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
157         # measure it against the Gemini path, or to avoid the cloud entirely). With
158         # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
159         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
160         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
161         if skip_ai:
162             model_name = "local-noai"
163             logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
164                       "be emitted as rallies; no %s handoff, no API key needed.",
165                       label, provider_name)
166         else:
167             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
168             api_key_env_var = f"{provider_name.upper()}_API_KEY"
169             api_key = os.environ.get(api_key_env_var)
170             if not api_key:
171                 from backend.pipeline.segmenters._keyhint import api_key_hint
172                 logger.error(

```

```

173         f"{api_key_env_var} environment variable is not set! "
174         f"To run the {provider_name} handoff, set your API key. "
175         + api_key_hint(api_key_env_var)
176     )
177     return []
178     try:
179         provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
180     except KeyError as e:
181         logger.error(str(e))
182         return []
183
184     video_duration = get_video_duration(video_path)
185     if video_duration <= 0:
186         logger.error("Invalid video duration.")
187         return []
188     fps, frame_width = self._probe_fps_width(video_path)
189
190     # --- Runner config + windowing thresholds ---
191     # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
192     # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
193     # it delegates to the subclass _build_runner (the real WSL/native runner).
194     try:
195         runner, runner_params = self._resolve_runner(idx_cfg)
196     except NotImplementedError as e:
197         # e.g. detector_impl="native" for a family without a native runner
198         # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
199         # crashing.
200         logger.error("%s: %s", label, e)
201         return []
202
203     velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
204     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
205     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
206     # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
207     max_window_duration = idx_cfg.get("max_window_duration", 0.0)
208     window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
209     window_hard_cap = idx_cfg.get("window_hard_cap", False)
210     window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
211     inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
212     inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
213     window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
214     window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
215     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
216     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
217     log_candidates = idx_cfg.get("log_yolo_candidates", True)
218     store_candidates = idx_cfg.get("store_yolo_candidates", True)
219
220     detection_params = {
221         "detector": self.name,
222         "velocity_thresh": velocity_thresh,
223         "merge_gap": merge_gap,
224         "min_action_window_duration": min_window_duration,
225         **runner_params,
226     }
227
228     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
229     ok, msg = runner.healthcheck()
230     if not ok:
231         logger.error("%s detector environment not ready: %s", label, msg)
232         return []
233     logger.info("%s detector env OK: %s", label, msg)

```

```

234 # --- Phase 2: trajectory -> candidate rally windows ---
235 # Trajectory detectors process the whole video in one pass (they carry
236 # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
237 t_start = time.time()
238 out_dir = os.path.join(output_base(self.config), self.family)
239 csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
240 if not csv_path:
241     logger.error("%s produced no trajectory; aborting.", label)
242     return []
243
244 points = runner.parse_trajectory_csv(csv_path)
245 logger.info("Parsed %d trajectory points (%d visible).",
246            len(points), sum(1 for p in points if p.visible))
247
248 all_candidate_windows = runner.trajectory_to_action_windows(
249     points, fps=fps, frame_width=frame_width, chunk_start=0.0,
250     velocity_thresh=velocity_thresh, merge_gap=merge_gap,
251     min_window_duration=min_window_duration, chunk_index=0,
252     max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
253     window_hard_cap=window_hard_cap,
254     window_inactivity_end=window_inactivity_end,
255     inactivity_rally_thresh=inactivity_rally_thresh,
256     inactivity_min_density=inactivity_min_density,
257     window_blob_fallback=window_blob_fallback,
258     window_blob_factor=window_blob_factor,
259 )
260 logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
261            label, time.time() - t_start, len(all_candidate_windows))
262
263 if log_candidates:
264     for cw in all_candidate_windows:
265         logger.info(f"[{label} rally]
266 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
267                    f"({cw['end'] - cw['start']:.1f}s) |
268 frames={cw['active_frame_count']} "
269                    f"peak_v={cw['peak_velocity']:.3f}
270 mean_v={cw['mean_velocity']:.3f}")
271
272 # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
273 fusion
274 # segmenter adds net_crossings + person-cue features. Computed once, reused for
275 both
276 # persistence and the handoff gate below.
277 window_stats = [
278     self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
279     idx_cfg)
280     for cw in all_candidate_windows
281 ]
282
283 # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
284 candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
285 if store_candidates:
286     for i, cw in enumerate(all_candidate_windows):
287         candidate_ids[i] = self.db.add_candidate_segment(
288             video_id, detector=self.name,
289             start_time=cw["start"], end_time=cw["end"],
290             chunk_index=cw["chunk_index"], confidence=min(1.0,
291             cw["peak_velocity"]),
292             stats=window_stats[i], detection_params=detection_params,
293         )
294
295 if not all_candidate_windows:
296     return []
297
298 # --- Phase 3: AI provider handoff over padded candidate clips ---

```

```

292         # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
293         # candidates above stay the full superset ? only the served play_segments are
gated.
294         handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
295
296         ai_segments: List[dict] = []
297         if skip_ai:
298             # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
299             # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
300             # falls back to the duration-based estimate (no AI exchange count).
301             ai_segments = [
302                 {
303                     "start_time": all_candidate_windows[wi]["start"],
304                     "end_time": all_candidate_windows[wi]["end"],
305                     "shuttle_exchange_count": None,
306                     "shot_count_confidence": "LocalNoAI",
307                     "_candidate_id": candidate_ids[wi],
308                 }
309                 for wi in handoff_indices
310             ]
311             logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
312                         "(no AI handoff).", len(ai_segments))
313         else:
314             handoff_dir = os.path.join(output_base(self.config),
f"chunks_{provider_name}_handoff")
315             os.makedirs(handoff_dir, exist_ok=True)
316             logger.info("Phase 3: %s validation on %d candidate windows...",
317                         provider_name, len(handoff_indices))
318
319             def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
320                 w_start = max(0, window["start"] - handoff_padding)
321                 w_end = min(video_duration, window["end"] + handoff_padding)
322                 w_dur = w_end - w_start
323                 w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
324                 if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
325                     return []
326                 prompt = sport_profile.get_prompt(chunk_duration=w_dur)
327                 segments, _ = provider.analyze_video(w_path, prompt,
328                                                     label=f"Handoff {wi+1}")
329                 valid = []
330                 for seg in segments:
331                     try:
332                         seg["start_time"] = float(seg["start_time"]) + w_start
333                         seg["end_time"] = float(seg["end_time"]) + w_start
334                         seg["_candidate_id"] = candidate_ids[wi]
335                         valid.append(seg)
336                     except (ValueError, TypeError, KeyError) as e:
337                         logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
338                 try:
339                     os.remove(w_path)
340                 except OSError:
341                     pass
342                 return valid
343
344             with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
345                 futures = [executor.submit(process_candidate_window, wi,

```

```

all_candidate_windows[wi])
346             for wi in handoff_indices]
347         for future in futures:
348             ai_segments.extend(future.result())
349         try:
350             os.rmdir(handoff_dir)
351         except OSError:
352             pass
353         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
354
355     # --- Phase 4: DB population + candidate linking ---
356     segments_data = []
357     for segment in ai_segments:
358         try:
359             start = float(segment["start_time"])
360             end = float(segment["end_time"])
361             dur = end - start
362             metadata = {
363                 "shot_count": self._shot_count_from(segment, dur),
364                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
365                 "detector": f"{self.family}-hybrid-{model_name}",
366             }
367             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
368             if segment_id:
369                 candidate_id = segment.get("_candidate_id")
370                 if candidate_id is not None:
371                     self.db.link_candidate_to_segment(candidate_id, segment_id)
372                 segments_data.append({
373                     "id": segment_id, "start_time": start, "end_time": end,
374                     "duration": dur, "metadata": metadata,
375                 })
376         except Exception as e:
377             logger.warning("Skipping invalid segment struct: %s", e)
378
379     return segments_data
380

```

11. assistant (2026-06-22T09:49:26.352Z)

Now let me verify the critical finding by examining the trajectory_hybrid.py file around lines 138-143 and the return statement around line 379.

12. user (2026-06-22T09:49:28.052Z)

```

135         re-scoring."""
136         return list(range(len(windows)))
137
138     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
139                     player_pool: Optional[List[str]] = None,
140                     max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
141         # override-ignore: the ABC declares the Result contract (Success|Failure)
142         # but every live segmenter still returns a bare list ? the documented
143         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
144         label = self.display_label

```

13. user (2026-06-22T09:49:29.391Z)

```

375         })
376     except Exception as e:
377         logger.warning("Skipping invalid segment struct: %s", e)
378

```

```
379         return segments_data
380
```

14. user (2026-06-22T09:49:29.872Z)

```
1     from abc import ABC, abstractmethod
2     from typing import List, Dict, Any, Type, Optional
3     from backend.infrastructure.database import Database
4     from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6     class BasePlaySegmenter(ABC):
7         def __init__(self, db: Database, config: Dict[str, Any]):
8             self.db = db
9             self.config = config
10
11         @property
12         @abstractmethod
13         def name(self) -> str:
14             """Unique identifier for this segmenter."""
15             pass
16
17         @property
18         @abstractmethod
19         def description(self) -> str:
20             """Human-readable explanation of this segmenter."""
21             pass
22
23         @abstractmethod
24         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25             """
26             Processes a video to detect play segments, populates SQLite database,
27             and returns a Result.
28
29             Use isinstance(result, Failure) (or result.is_success()) to distinguish
30             "no segments found" from "something went wrong" (missing key, healthcheck
31             failure, etc.). This is the start of the standardized error/result contract.
32             """
33             pass
34
35     class SegmenterRegistry:
36         def __init__(self):
37             self.segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39         def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40             """Registers a new segmenter class under its name property."""
41             try:
42                 temp_inst = segmenter_cls(None, {})
43                 name = temp_inst.name
44             except Exception as e:
45                 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46             self.segmenters[name] = segmenter_cls
47
48         def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49             return self.segmenters.get(name)
50
51         def list_available(self) -> Dict[str, str]:
52             """Returns a dict of registered segmenter names and their descriptions."""
53             res = {}
54             for name, cls in self.segmenters.items():
55                 try:
56                     temp_inst = cls(None, {})
57                     desc = temp_inst.description
```

```

58         except Exception:
59             desc = "No description available."
60             res[name] = desc
61         return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

15. assistant (2026-06-22T09:49:32.247Z)

Now let me check how nightly_reelcount.py actually handles the result from process_video:

16. user (2026-06-22T09:49:34.181Z)

```

340     def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341         random.seed(0) # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342         db = Database(db_path)
343         video_id = compute_video_id(local)
344         t0 = time.monotonic()
345         result = seg_cls(db, config).process_video(video_id, local)
346         wall = time.monotonic() - t0
347         if isinstance(result, Failure):
348             return None, None, wall, getattr(result, "message", "segmenter failed")
349         segs = result.value if hasattr(result, "value") else result
350         intervals = _segments_to_intervals(segs)
351         return len(segs), intervals, wall, None
352
353     import tempfile
354     total_wall = 0.0
355     with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356         count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357         total_wall += wall
358         if err is not None:
359             return {"name": name, "reel_count": None, "ok": False, "error": err,

```

17. user (2026-06-22T09:49:34.931Z)

Found 9 files
backend\ui\assets\index-GSXqL6iY.js
backend\results.py
backend\pipeline\validators\base.py
backend\eval\served_gate_a.py
backend\eval\regression.py
backend\eval\promotion.py
backend\eval\per_user_regression.py
backend\eval\metrics.py
backend\compute\base.py

18. assistant (2026-06-22T09:49:37.365Z)

Let me read the results.py to understand the Result contract:

19. user (2026-06-22T09:49:37.868Z)

```

1     """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3     This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5     The goal is to make it impossible to confuse "no data / no rallies" with "something
actually failed"
6     (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
7

```

```

8     We use simple dataclasses so callers can use isinstance checks or structural pattern
matching.
9     This is intentionally small and incremental ? we start by applying it to the segmenter
10    and provider layers (the biggest sources of silent [] returns today).
11
12    The types are generic where it makes sense. Callers can still access .value on Success
13    or .message on Failure.
14
15    This module has no dependencies on the rest of the app.
16    """
17
18    from __future__ import annotations
19
20    from dataclasses import dataclass, field
21    from typing import Generic, TypeVar, List, Optional, Any, Tuple, Union
22
23    T = TypeVar("T")
24
25
26    @dataclass
27    class Success(Generic[T]):
28        """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30        value: T
31        warnings: List[str] = field(default_factory=list)
32
33        def is_success(self) -> bool:
34            return True
35
36        def unwrap(self) -> T:
37            """Return the value. Only safe to call if you already know it's a Success."""
38            return self.value
39
40
41    @dataclass
42    class Failure:
43        """Represents a failure with a human-readable message and optional details."""
44
45        message: str
46        details: Optional[dict[str, Any]] = None
47        is_recoverable: bool = False # e.g. missing API key (recoverable) vs internal crash
48
49        def is_success(self) -> bool:
50            return False
51
52        def unwrap(self) -> Any:
53            """Raise to make misuse obvious in code that assumes success."""
54            raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57    # Convenient alias for use in type hints: Result[list[dict], Failure]
58    # Result[ValueType] means Success[ValueType] | Failure
59    # (the Failure "error" type is fixed for this contract)
60    Result = Union[Success[T], Failure]
61
62
63    def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
64        """Split a segmenter/provider return into ``(value, failure)`` so a caller branches
once.
65
66        - ``Failure``          -> ``(None, failure)``
67        - ``Success(value=v)`` -> ``(v, None)``
68        - a bare value (legacy code that predates this contract and still returns a raw
list)
69
        -> ``(value, None)``

```

```

70
71         This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline;
callers
72         that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
len()``
73         on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
heterogeneous
74         (Success | Failure | legacy bare value)."""
75         if isinstance(result, Failure):
76             return None, result
77         value = result.value if hasattr(result, "value") else result
78         return value, None
79

```

20. assistant (2026-06-22T09:49:40.394Z)

Excellent. Now let me check if there's any other segmenter implementation to see if the contract gap is universal:

21. user (2026-06-22T09:49:41.676Z)

```

C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/base.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/fusion_hybrid.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/gemini.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/motion.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/tracknet_hybrid.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/trajectory_hybrid.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/wasb_hybrid.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/yolo_hybrid.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/_keyhint.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/__init__.py

```

22. assistant (2026-06-22T09:49:43.702Z)

Let me check motion.py and wasb_hybrid.py to see their return patterns:

23. user (2026-06-22T09:49:44.689Z)

```

20:     def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None,
max_frames: int = None) -> List[Dict[str, Any]]:

```

24. user (2026-06-22T09:49:47.445Z)

```

15
16         @property
17         def description(self) -> str:
18             return "Traditional CV pixel-motion heuristics tracking frame differences."
19
20         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
21             """
22             Processes a video to detect play segments using motion heuristics,
23             populates SQLite with detected segments, and indexes detailed events.
24             """

```

```

25         if not player_pool:
26             player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
27
28         cap = cv2.VideoCapture(video_path)
29         if not cap.isOpened():
30             logger.error(f"Segmenter could not open video: {video_path}")
31             return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
32
33         final_segments = self._detect_motion_segments(cap, max_frames)
34
35         segments_data = self._populate_db_with_events(video_id, final_segments,
player_pool)
36
37         return Success(value=segments_data)
38
39     def _detect_motion_segments(self, cap: Any, max_frames: int = None) ->
List[Tuple[float, float]]:
40         """Run the pixel-motion heuristic over ``cap`` and return the detected
41         (start, end) play segments.
42
43         Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
44         thresholds it into raw segments, then merges dead-time gaps and filters
45         out segments shorter than ``min_segment_duration``. ``cap`` is released
46         before returning. This is the detection half of ``process_video`` ? it
47         performs no DB writes and no fabrication.
48         """
49         fps = cap.get(cv2.CAP_PROP_FPS)
50         total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
51         duration = total_frames / fps if fps > 0 else 0
52
53         # Sub-sample settings (5 FPS for analysis)
54         analysis_fps = 5.0
55         frame_interval = max(1, int(fps / analysis_fps))
56
57         idx_cfg = self.config.get("indexing", {})
58         motion_threshold = idx_cfg.get("motion_threshold", 0.08)
59         min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
60         dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
61
62         prev_gray = None
63         motion_signals = []
64         timestamps = []
65
66         frame_idx = 0
67         processed_count = 0
68         t_start = time.time()
69         # Emit a progress line roughly every 5% of the video so long runs are
70         # observable instead of silent (a full 1080p match is ~tens of minutes).
71         progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
72         next_progress = progress_step
73         logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
74             f"({~{duration/60:.1f} min), analyzing every {frame_interval}th
frame")
75         while True:
76             # Skip decoding for intermediate frames using cap.grab() for high
performance
77             for _ in range(frame_interval - 1):
78                 if not cap.grab():
79                     break
80                 frame_idx += 1
81
82             ret, frame = cap.read()
83             if not ret:
84                 break

```

```

85
86         # Process frame
87         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
88         # Apply Gaussian Blur to smooth noise
89         gray = cv2.GaussianBlur(gray, (21, 21), 0)
90
91         t = frame_idx / fps
92         timestamps.append(t)
93
94         if prev_gray is not None:

```

25. assistant (2026-06-22T09:49:49.787Z)

Perfect! Now I have the concrete evidence. Let me check wasb_hybrid.py's process_video return:

26. user (2026-06-22T09:49:51.612Z)

```

29:         return "wasb_hybrid"
33:         return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
38:         return WasbRunner(cfg), {"weights_path": cfg.weights_path}
45:         return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,

```

27. assistant (2026-06-22T09:49:53.766Z)

Let me check if wasb_hybrid inherits from trajectory_hybrid:

28. user (2026-06-22T09:49:54.485Z)

```

1         """WASB-based hybrid play segmenter.
2
3         Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4         WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5         badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7         For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8         candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9         min_window_duration) are what get tuned against golden rally labels.
10
11         The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12         lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13         WASB runner and identity.
14         """
15         from typing import Any, Dict, Tuple
16
17         from backend.pipeline.segmenters.base import segmenter_registry
18         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19         from backend.pipeline.detectors.base import DetectorRunner
20         from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23         class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24             family = "wasb"
25             display_label = "WASB"
26
27             @property
28             def name(self) -> str:
29                 return "wasb_hybrid"
30
31             @property
32             def description(self) -> str:
33                 return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
34                 "weights) for precise candidate rallies + AI provider for semantic

```

```

boundaries.")
35
36         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
37             cfg = WasbConfig.from_indexing_cfg(idx_cfg)
38             return WasbRunner(cfg), {"weights_path": cfg.weights_path}
39
40         def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
41             # Lazy import: the native runner pulls no heavy deps, but keep parity with the
stub
42             # seam (imported only when detector_impl='native' is actually selected).
43             from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
44             cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
45             return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
46                                             "detector_impl": "native", "device": cfg.device}
47
48
49     segmenter_registry.register(WasbHybridSegmenter)
50

```

29. assistant (2026-06-22T09:49:56.667Z)

Perfect! Now let me check how main.py handles the return values to confirm the gap is truly masked:

30. user (2026-06-22T09:49:58.083Z)

```

57:from backend.results import Failure # standardized error/result contract
513:         result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
516:         # state (same destroy-after-confirm contract as the Failure branch), then let
522:         if isinstance(result, Failure):
534:         segments = result.value if hasattr(result, "value") else result
541:         "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
560:def process_video(req: ProcessRequest, request: Request):
1051:         result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
1054:         if isinstance(result, Failure):
1059:         segments = result.value if hasattr(result, "value") else result

```

31. assistant (2026-06-22T09:50:00.124Z)

Perfect. Now I have all the evidence. Let me compile the final structured report.

32. user (2026-06-22T09:50:14.674Z)

Structured output provided successfully

33. assistant (2026-06-22T09:50:23.430Z)

Summary

The finding is **REAL** and **CONFIRMED**, but it is **documented-deliberate**, not a hidden defect:

What I found

trajectory_hybrid.py:138-143 literally admits in its docstring: "the ABC declares the

Result contract (Success|Failure) but every live segmenter still returns a bare list ? the documented deliberate-incremental gap (CODE_MAP gotchas; main.py handles both)."

The return at **line 379** is ``return segments_data`` ? a bare ``List[Dict]``, never wrapped in ``Success|Failure``.

Concrete contract violation

- **ABC (base.py:24):** `process_video` must return ``Result[List[Dict]]`` = ``Success | Failure``
- **motion.py:37:** correctly returns ``Success(value=segments_data)``
- **trajectory_hybrid.py:379:** returns raw ``segments_data`` (bare list)
- **wasb_hybrid.py + tracknet_hybrid.py:** inherit from `trajectory_hybrid`, so they both emit bare lists

How it's masked (safe for nightly)

The nightly runner (lines 347-349) and `main.py` (lines 534, 1059) both have defensive fallback handling:

```
```python
if isinstance(result, Failure):
 ...
segs = result.value if hasattr(result, "value") else result # ? catches bare lists
```
```

This graceful fallback makes the type system lie, but the `nightly_reelcount.py` regression run is safe ? it handles both Success-wrapped and bare-list returns.

Recommendation

- **No action for `nightly_reelcount.py`** ? it's already defended and working.
- **Long-term debt:** Migrate `trajectory_hybrid.py` to return ``Success(value=segments_data)`` and remove the `hasattr` fallback (low priority, documented as intentional).